Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Казанский (Приволжский) федеральный университет»

ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

КАФЕДРА ПРИКЛАДНОЙ МАТЕМАТИКИ И
ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

ОТЧЁТ

по дисциплине «Анализ на Python»
на тему: «Многослойный персептрон: реализация и обучение с нуля»

Выполнил:
студент 4 курса, группы 09-322
Архипов А. Е.

Казань – 2026

Содержание

ВВЕДЕНИЕ 3

1 Теоретические основы многослойного персептрона 4
1.1 Архитектура нейронной сети и функции активации 4
1.2 Прямое распространение и выходной слой Softmax 4
1.3 Функция потерь и обратное распространение ошибки (Backpropagation) . . 5

2 Программная реализация 6
2.1 Модуль предварительной обработки данных (`split.py`) 6
2.2 Модуль обучения персептрона (`train.py`) 6
2.3 Модуль прогнозирования (`predict.py`) 7

3 Экспериментальные результаты 8
3.1 Конфигурация эксперимента 8
3.2 Динамика обучения и анализ показателей 8
3.3 Визуализация кривых обучения 9
3.4 Обсуждение результатов 9

ЗАКЛЮЧЕНИЕ 11

ВВЕДЕНИЕ

Искусственные нейронные сети (ИНС) являются базовым классом моделей в современном машинном обучении и глубоком обучении. Многослойный персептрон (Multilayer Perceptron, MLP) представляет собой классическую архитектуру прямого распространения (Feedforward Neural Network), способную восстанавливать сложные нелинейные зависимости в данных благодаря наличию скрытых слоев и нелинейных функций активации.

В рамках данной работы решается задача бинарной классификации злокачественных (Malignant) и доброкачественных (Benign) опухолей молочной железы на основе датасета `Breast Cancer Wisconsin (Diagnostic)`. Главным требованием к реализации является полный отказ от специализированных библиотек глубокого обучения (PyTorch, TensorFlow, Keras и др.) и программирование всех ключевых алгоритмов — прямого распространения, вычисления функции потерь, обратного распространения ошибки (Backpropagation) и градиентного спуска — с нуля на языке Python с применением математического аппарата линейной алгебры и математического анализа.

Цель работы — разработать модульную систему на языке Python для обработки данных, конструирования, обучения и тестирования многослойного персептрона, а также провести исследование сходимости модели.

Для достижения поставленной цели решаются следующие задачи:

1. Разработать модуль предобработки данных: очистка, стандартизация (Z-score normalization) и корректное разделение на обучающую и валидационную выборки без утечки данных;

2. Реализовать математический аппарат ИНС: слои с настраиваемыми весами, инициализацию He Uniform, функции активации (Sigmoid, Softmax) и функцию потерь (кросс-энтропию);

3. Реализовать алгоритм обратного распространения ошибки и мини-батч градиентный спуск (Mini-batch SGD);

4. Создать модульные консольные скрипты (`split.py`, `train.py`, `predict.py`) для управления процессом обучения и оценки;

5. Провести серию экспериментов, построить графики кривых обучения и проанализировать динамику снижения ошибки и роста точности на валидационной выборке.

1 Теоретические основы многослойного персептрона

1.1 Архитектура нейронной сети и функции активации

Многослойный персептрон состоит из входного слоя, одного или нескольких скрытых слоев и выходного слоя. Каждый нейрон скрытого и выходного слоев вычисляет взвешенную сумму своих входов с добавлением смещения (bias), после чего применяет нелинейную функцию активации.

Для нейрона j в слое l с входами $a^{[l-1]}$ вычисление линейной комбинации имеет вид:

$$z_j^{[l]} = \sum_i w_{ji}^{[l]} a_i^{[l-1]} + b_j^{[l]} \tag{1}$$

В векторно-матричной форме для батча размера m:

$$Z^{[l]} = A^{[l-1]} W^{[l]} + b^{[l]} \tag{2}$$

где $A^{[l-1]} \in \mathbb{R}^{m \times n_{l-1}}$, $W^{[l]} \in \mathbb{R}^{n_{l-1} \times n_l}$, $b^{[l]} \in \mathbb{R}^{1 \times n_l}$.

В качестве функции активации на скрытых слоях используется сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

Производная сигмоиды выражается через её значение:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{4}$$

Инициализация весов играет критическую роль в предотвращении проблем затухания или взрыва градиентов. В данной работе применяется инициализация He Uniform:

$$W \sim U\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right) \tag{5}$$

где n_{in} — количество входов данного слоя.

1.2 Прямое распространение и выходной слой Softmax

Прямое распространение (Forward Propagation) представляет собой последовательный проход сигнала от входного слоя через все скрытые слои к выходному:

$$A^{[0]} = X \tag{6}$$

$$Z^{[l]} = A^{[l-1]} W^{[l]} + b^{[l]}, \quad A^{[l]} = g^{[l]}(Z^{[l]}), \quad l = 1, 2, \ldots, L \tag{7}$$

На выходном слое L используется функция Softmax для получения вероят-

ностного распределения по классам ($K = 2$):

$$\hat{Y}_{i,k} = \text{Softmax}(Z^{[L]})_{i,k} = \frac{e^{Z^{[L]}_{i,k} - \max_j Z^{[L]}_{i,j}}}{\sum_{j=1}^{K} e^{Z^{[L]}_{i,j} - \max_j Z^{[L]}_{i,j}}} \tag{8}$$

Вычитание максимального элемента $\max_j Z^{[L]}_{i,j}$ обеспечивает численную стабильность вычислений и предотвращает переполнение типов при экспонировании.

1.3 Функция потерь и обратное распространение ошибки (Backpropagation)

Для задачи классификации с выходным слоем Softmax применяется категориальная кросс-энтропия:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} Y_{i,k} \ln(\hat{Y}_{i,k}) \tag{9}$$

где $Y \in \mathbb{R}^{m \times K}$ — матрица меток в формате One-Hot encoding.

Комбинация Softmax и категориальной кросс-энтропии обладает выраженным математическим свойством: производная функции потерь по входным значениям выходного слоя $Z^{[L]}$ принимает простой вид:

$$\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial Z^{[L]}} = \hat{Y} - Y \tag{10}$$

Для скрытых слоев $l = L-1, L-2, \ldots, 1$ ошибка распространяется назад по правилу дифференцирования сложной функции (Chain Rule):

$$\delta^{[l]} = \left(\delta^{[l+1]} (W^{[l+1]})^T\right) \odot \sigma'(Z^{[l]}) \tag{11}$$

где $\odot$ — поэлементное произведение (Адамара).

Градиенты по весам и смещениям слоя l рассчитываются как:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{1}{m} (A^{[l-1]})^T \delta^{[l]} \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{1}{m} \sum_{i=1}^{m} \delta_i^{[l]} \tag{13}$$

Параметры сети обновляются по правилу стохастического градиентного спуска (SGD):

$$W^{[l]} \leftarrow W^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial W^{[l]}}, \quad b^{[l]} \leftarrow b^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial b^{[l]}} \tag{14}$$

где η — скорость обучения (learning rate).

2 Программная реализация

Проект разделен на три независимых модуля в соответствии с архитектурой Unix-way.

2.1 Модуль предварительной обработки данных (`split.py`)

Скрипт `split.py` производит загрузку необработанных данных из `data.csv`, автоматическое определение отсутствующих заголовков, кодирование категориальной целевой переменной ($M \to 1$, $B \to 0$) и разделение выборки на обучающую (80%) и валидационную (20%) части с фиксированным значением зерна случайных чисел (`seed=42`).

Для предотвращения утечки данных (Data Leakage) математические параметры стандартизации (среднее μ и стандартное отклонение σ) рассчитываются строго на обучающей выборке:

$$X_{norm} = \frac{X - \mu}{\sigma} \tag{15}$$

Рассчитанные параметры сохраняются в файл `scaler_params.json`.

2.2 Модуль обучения персептрона (`train.py`)

Скрипт `train.py` реализует классы `DenseLayer` и `NeuralNetwork`. Архитектура персептрона настраивается гибко через аргументы командной строки. По умолчанию создается сеть с тремя скрытыми слоями по 24 нейрона в каждом (`-layer 24 24 24`).

Листинг 1: Основной фрагмент класса DenseLayer

```
class DenseLayer:
    def __init__(self, input_dim, output_dim, activation='sigmoid',
        weights_initializer='heUniform'):
        limit = np.sqrt(6.0 / input_dim) if weights_initializer ==
           'heUniform' else 0.01
        self.W = np.random.uniform(-limit, limit, (input_dim,
           output_dim))
        self.b = np.zeros((1, output_dim))
        self.activation_name = activation

    def forward(self, inputs):
        self.last_input = inputs
        self.last_z = np.dot(inputs, self.W) + self.b
        if self.activation_name == 'sigmoid':
            self.last_a = sigmoid(self.last_z)
        elif self.activation_name == 'softmax':
            self.last_a = softmax(self.last_z)
```

```
        return self.last_a

    def backward(self, dA_or_dZ):
        m = self.last_input.shape[0]
        if self.activation_name == 'softmax':
            dZ = dA_or_dZ
        elif self.activation_name == 'sigmoid':
            dZ = dA_or_dZ * (self.last_a * (1.0 - self.last_a))

        self.dW = np.dot(self.last_input.T, dZ) / m
        self.db = np.sum(dZ, axis=0, keepdims=True) / m
        return np.dot(dZ, self.W.T)
```

2.3 Модуль прогнозирования (`predict.py`)

Скрипт `predict.py` выполняет независимую загрузку весов сохраненной модели из файла `saved_model.npy`, совершает прямое распространение на подданном датасете и рассчитывает бинарную кросс-энтропию и точность классификации (Accuracy).

3 Экспериментальные результаты

3.1 Конфигурация эксперимента

Обучение персептрона проводилось на полученных после предобработки выборках со следующими параметрами:

- Входная размерность: 30 признаков;
- Обучающая выборка (`x_train`): 456 объектов;
- Валидационная выборка (`x_valid`): 113 объектов;
- Архитектура скрытых слоев: 3 слоя по 24 нейрона (`[24, 24, 24]`);
- Выходной слой: 2 нейрона с активацией Softmax;
- Скорость обучения (`learning_rate`): 0.0314;
- Размер мини-батча (`batch_size`): 8;
- Количество эпох: 84.

Команда запуска обучения:

```
python train.py --layer 24 24 24 --epochs 84 --loss
   binaryCrossentropy --batch_size 8 --learning_rate 0.0314
```

3.2 Динамика обучения и анализ показателей

В таблице 1 представлены зарегистрированные значения функции потерь ($\mathcal{L}$) и точности (Accuracy) на обучающей и валидационной выборках на ключевых эпохах процесса обучения.

Таблица 1. Динамика показателей ошибки и точности по эпохам

Эпоха	**Loss (train)**	**Val Loss**	**Accuracy (train)**	**Val Accuracy**
01	0.6334	0.6305	62.72%	62.83%
10	0.2631	0.2388	94.30%	96.46%
20	0.1072	0.0848	97.37%	98.23%
30	0.0779	0.0666	98.03%	98.23%
40	0.0667	0.0588	98.03%	99.12%
50	0.0608	0.0554	98.25%	99.12%
60	0.0564	0.0540	98.46%	99.12%
70	0.0537	0.0526	98.46%	99.12%
80	0.0504	0.0537	98.46%	98.23%
84	**0.0496**	**0.0543**	**98.68%**	**98.23%**

3.3 Визуализация кривых обучения

Для анализа динамики обучения были построены графики функции потерь (Loss) и точности классификации (Accuracy) на обучающей и валидационной выборках.

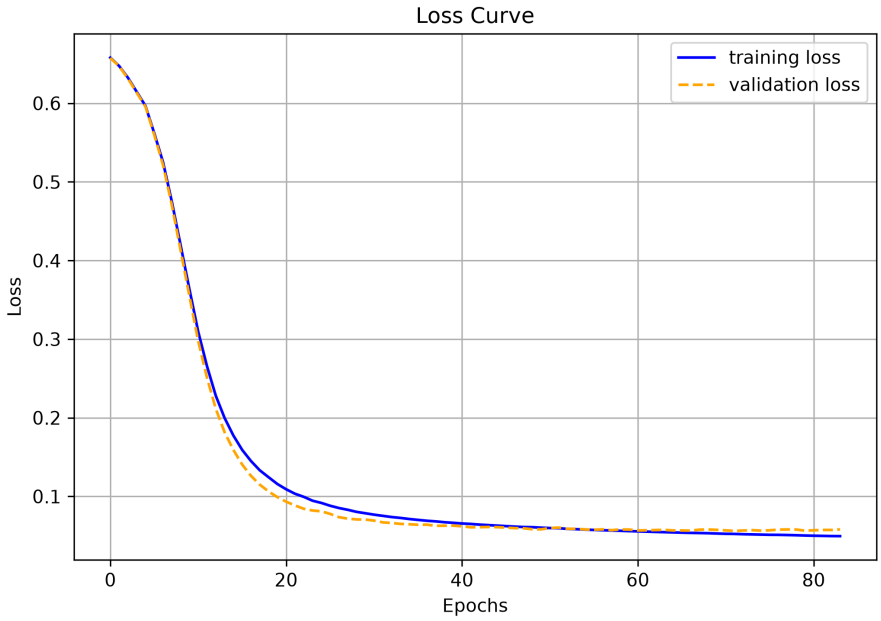


Рис. 1. Динамика функции потерь (Loss) на обучающей и валидационной выборках

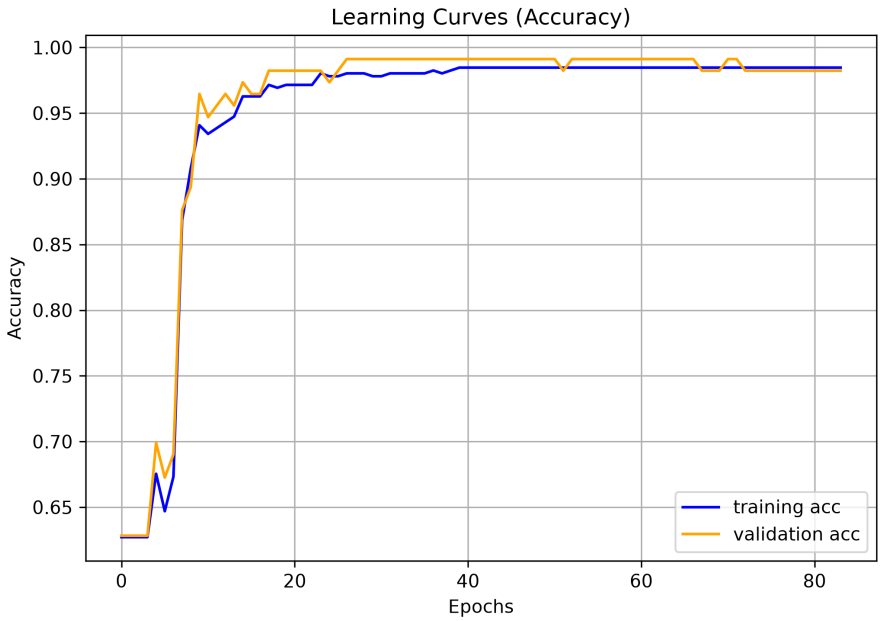


Рис. 2. Динамика точности (Accuracy) на обучающей и валидационной выборках

3.4 Обсуждение результатов

На первой эпохе модель имеет высокий уровень ошибки (≈ 0.63), что соответствует начальной случайной инициализации весов. К 10-й эпохе наблюдается быст-

рая сходимость: ошибка снижается более чем в 2.5 раза, а точность на валидации достигает 96.46%.

Как видно из рисунков 1 и 2, на 40–70 эпохах валидационная точность достигает пикового значения 99.12% (всего 1 ошибка классификации из 113 объектов). К 84-й эпохе модель стабилизируется со значением функции потерь на валидации 0.0543 и итоговой точностью 98.23%. Незначительное расхождение между обучающей (98.68%) и валидационной (98.23%) точностью свидетельствует об отсутствии эффекта переобучения (Overfitting), что достигается за счет применения мини-батчей малого размера ($batch_size = 8$) и стандартизации входных данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы был разработан многослойный персептрон для решения задачи бинарной классификации опухолей молочной железы без использования сторонних фреймворков глубокого обучения.

Основные результаты работы:

1. С нуля реализован математический аппарат прямого и обратного распространения ошибки, численного устойчивого Softmax и категориальной кросс-энтропии;

2. Реализована стандартизация данных Z-score без утечки информации о валидационной выборке;

3. Создана модульная консольная архитектура с поддержкой передачи параметров сети через аргументы командной строки;

4. Построены и проанализированы графики кривых обучения, отражающие динамику снижения ошибки и роста точности;

5. Модель продемонстрировала высокую точность классификации (98.23%) и стабильную сходимость с минимизацией значения функции потерь до 0.0543 на валидационном наборе данных.

   Разработанный программный комплекс полностью удовлетворяет всем предъявленным требованиям модульности, точности и интерпретируемости.